为什么 MySQL 选择使用 B+ 树作为索引结构？ - 面试鸭 | 2026最新面试题+详细答案解析

> ## Excerpt
>
> 2026最新后端、MySQL、数据库面试题，包含详细答案解析和练习题，23431+程序员已学习，已帮助10万+程序员通过校招/社招/实习面试，拿到offer！

回答重点

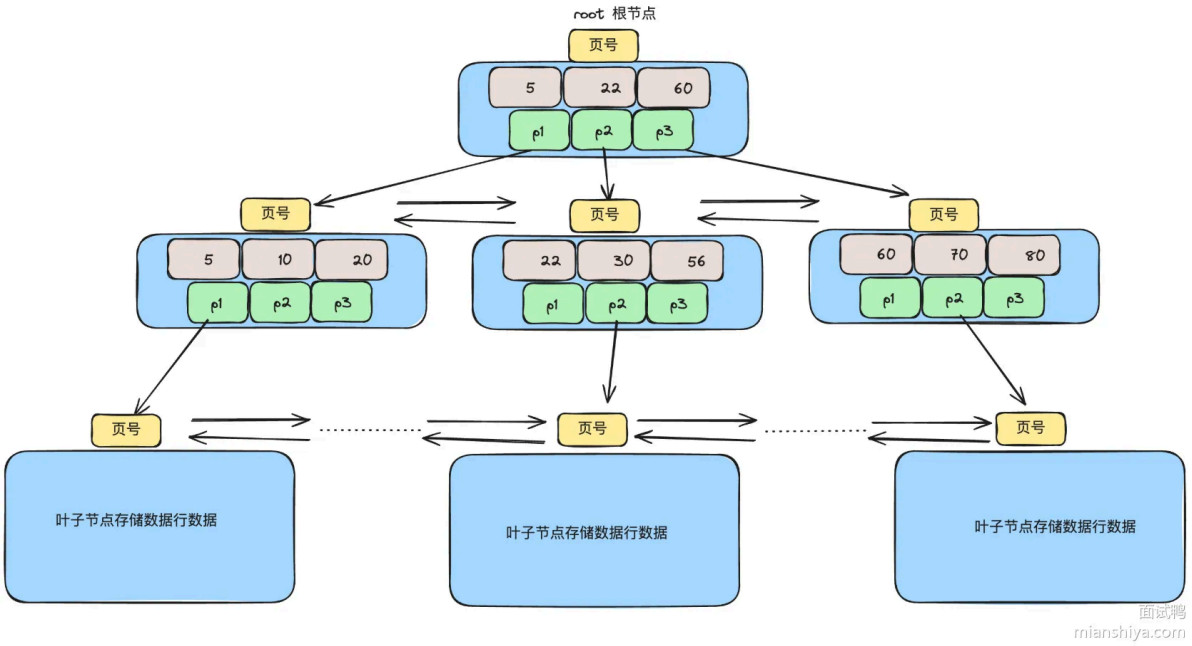
MySQL 选 B+ 树核心就一个原因：**磁盘 IO 次数最少**。数据库的数据存在磁盘上，磁盘随机读写比内存慢 10 万倍，所以索引结构的设计目标就是尽量减少磁盘访问次数。

B+ 树能做到这一点靠三个特性：

1）树矮。B+ 树是多叉树，一个节点能存几百上千个 key。3 层的 B+ 树就能存两千多万条数据，查任何一条最多 3 次磁盘 IO。红黑树是二叉树，存同样的数据量要二十多层，IO 次数直接爆炸。

2）非叶子节点只存 key 和指针，不存数据。一个 16KB 的页能塞进更多索引项，内存里能缓存更多索引，命中率高，磁盘访问少。

3）叶子节点用双向链表串起来。范围查询时，定位到起点后顺着链表往后扫就行，不用再回根节点重新查找，顺序 IO 比随机 IO 快太多。



扩展知识

为什么不用哈希表

哈希表等值查询 O(1)，看起来比 B+ 树的 O(log n) 快多了，但数据库场景下有两个致命问题：

第一，不支持范围查询。`where id > 100 and id < 200` 这种 SQL，哈希表只能全表扫描，B+ 树顺着叶子链表走一遍就完事。

第二，不支持排序。`order by` 得把数据捞出来再排，B+ 树的叶子节点天然有序，扫描出来就是排好的。

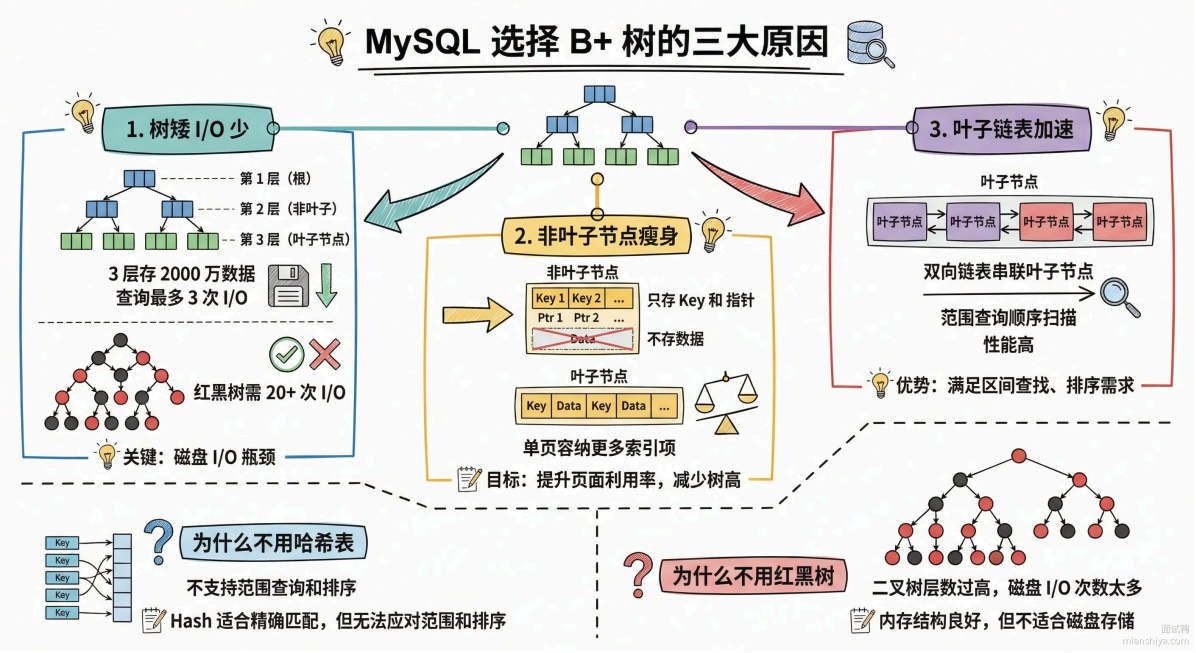
不过 InnoDB 也没完全放弃哈希，它有个自适应哈希索引。如果发现某个索引值被频繁访问，会自动在内存里建一个哈希索引加速查询。这个对业务透明，不需要手动创建。

为什么不用红黑树、AVL 树

红黑树和 AVL 树都是二叉树，每个节点最多两个子节点。数据量一大，树的高度蹭蹭往上涨。1000 万条数据，红黑树大概 23 层，查一条数据最坏情况要 23 次磁盘 IO。

B+ 树是多叉树，一个节点能存多少个 key 取决于页的大小和 key 的长度。InnoDB 默认页大小 16KB，假设主键是 bigint 占 8 字节，加上 6 字节的指针，一个非叶子节点能存 16KB / 14B ≈ 1170 个 key。三层 B+ 树：第一层 1 个节点，第二层 1170 个节点，第三层 1170 × 1170 ≈ 137 万个叶子节点。每个叶子节点假设存 16 条数据，三层就能存 2000 多万条数据，查任何一条最多 3 次 IO。

这个差距就是为什么内存数据库可以用红黑树，磁盘数据库必须用 B+ 树。



B+ 树和 B 树的区别

B 树也是多叉平衡树，和 B+ 树最大的区别是数据存储位置：

1） B 树每个节点都存完整数据，B+ 树只有叶子节点存数据，非叶子节点只存 key 和指针。这意味着 B+ 树的非叶子节点更小，一页能装更多索引项，树更矮。

2） B+ 树叶子节点用链表串起来，范围查询直接顺序扫描。B 树做范围查询要不断回到上层节点再往下找，IO 次数多。

3） B 树有可能在非叶子节点就找到数据直接返回，查询时间不稳定。B+ 树必须查到叶子节点，时间稳定可预期。

数据库选 B+ 树，文件系统有些用 B 树，比如 MongoDB 早期版本用的就是 B 树。

InnoDB 的 B+ 树实现细节

InnoDB 的索引分两种：聚簇索引和二级索引。

聚簇索引的叶子节点存的是完整的行数据，按主键排序。一张表只有一个聚簇索引，就是主键索引。如果没定义主键，InnoDB 会找一个唯一非空索引代替，都没有就自己生成一个 6 字节的 rowid。

二级索引的叶子节点存的是索引列的值 + 主键值。通过二级索引查数据时，先在二级索引树上找到主键，再回聚簇索引树上查完整数据，这就是回表。

页分裂和页合并

B+ 树的节点就是数据页，InnoDB 默认 16KB。数据插入时如果页满了，会触发页分裂：把一个页拆成两个，各存一半数据。删除数据时如果页的利用率太低，会触发页合并：把相邻两个页合成一个。

页分裂和页合并都涉及数据搬迁和指针修改，开销不小。所以自增主键比 UUID 主键好，自增主键总是往后追加，不会触发中间页的分裂。UUID 是随机的，插入位置也随机，频繁分裂影响性能。

面试官追问

提问：三层 B+ 树能存多少数据，你是怎么算的？

> 详细解答可以看：MySQL 三层 B+ 树能存多少数据？

回答：假设主键是 bigint 占 8 字节，页指针 6 字节，一个非叶子节点能存 16KB / 14B ≈ 1170 个 key。叶子节点存完整行，假设一行 1KB，一个叶子节点存 16 条。三层 B+ 树根节点指向 1170 个第二层节点，每个第二层节点指向 1170 个叶子，总共 1170 × 1170 ≈ 137 万个叶子，乘以每个叶子 16 条，大概 2000 多万条数据。行数据越小能存越多。

提问：为什么 MongoDB 早期用 B 树，后来又换成 B+ 树了？

> 详细解答可以看：B 树和 B+ 树相比有什么区别？

回答：MongoDB 早期定位是文档数据库，查询模式以单文档查询为主，B 树非叶子节点就能存数据，命中了直接返回，看起来更快。但后来发现范围查询和排序需求越来越多，B 树这方面太吃亏。WiredTiger 存储引擎引入后就换成了 B+ 树的变种，叶子节点也串起来了，范围查询性能大幅提升。

提问：自适应哈希索引是怎么工作的？什么时候该关掉它？

回答：InnoDB 监控索引页的访问模式，如果发现某个索引值被反复查询，就在内存里给它建个哈希索引。下次查同样的值直接哈希定位，跳过 B+ 树查找。这个功能对高并发点查场景有帮助，但它本身也消耗内存和 CPU，而且在高并发写的场景下，维护哈希索引的锁竞争反而成了瓶颈。如果监控发现 Adaptive Hash Index 的锁等待很高，可以考虑关掉。

提问：为什么 InnoDB 要求主键尽量短？

回答：两个原因。第一，主键越短，非叶子节点能存的 key 越多，树越矮，IO 次数越少。第二，所有二级索引的叶子节点都存了主键值，主键越长，二级索引占的空间越大。一个表有 5 个二级索引，主键从 int 换成 36 字节的 UUID，索引体积能膨胀好几倍。